Servomotoren der M17-Serie - DATENBLATT

Erschwingliche und einfache All-in-One-Bewegungssteuerung

Von der Ausbildung zur Innovation



Einführung

Die Servomotoren der M17-Serie sind All-in-One-Lösungen für die Bewegungssteuerung, die Motor, Motortreiber, Motion-Controller und Encoder in einem einzigen kompakten Gehäuse vereinen. Diese Servomotoren verfügen über eine RS-485-Kommunikationsschnittstelle, sodass mehrere Einheiten in Reihe geschaltet (Daisy-Chain) und über einen einzigen Anschlusspunkt gesteuert werden können. Die Serie ist in vier Modellen erhältlich - M17-60, M17-48, M17-40 und M17-34 - und bietet damit flexible Optionen für spezifische Drehmoment- und Größenanforderungen bei gleichbleibenden Regelungseigenschaften. Jeder Servomotor der M17-Serie bietet ausgereifte Steuerungsfunktionen, darunter mehrere Betriebsmodi, Selbstkalibrierung und integrierte Statusüberwachung über LED-Anzeigen. Die Motoren lassen sich einfach von jeder Plattform aus steuern, einschließlich Mac, PC, Raspberry Pi oder Arduino (es wird lediglich ein kostengünstiger RS485-Adapter benötigt), und eignen sich damit ideal für Ausbildungs- und Industrieanwendungen gleichermaßen. Die Serie unterstützt präzise Positionsregelung mit trapezförmigen Bewegungsprofilen, einen Closed-Loop-Regelmodus und eine umfassende Fehlerbehandlung. Mit standardisierten NEMA-17-Befestigungsmaßen, weitem Spannungsbereich (12-24V) und robustem Kommunikationsprotokoll bieten die Servomotoren der M17-Serie eine zuverlässige und flexible Lösung für Anwendungen, die eine präzise Bewegungssteuerung erfordern - von Robotik und CNC-Maschinen bis hin zu automatisierten Prüfeinrichtungen und wissenschaftlichen Instrumenten.

Senden Sie uns Ihr Feedback

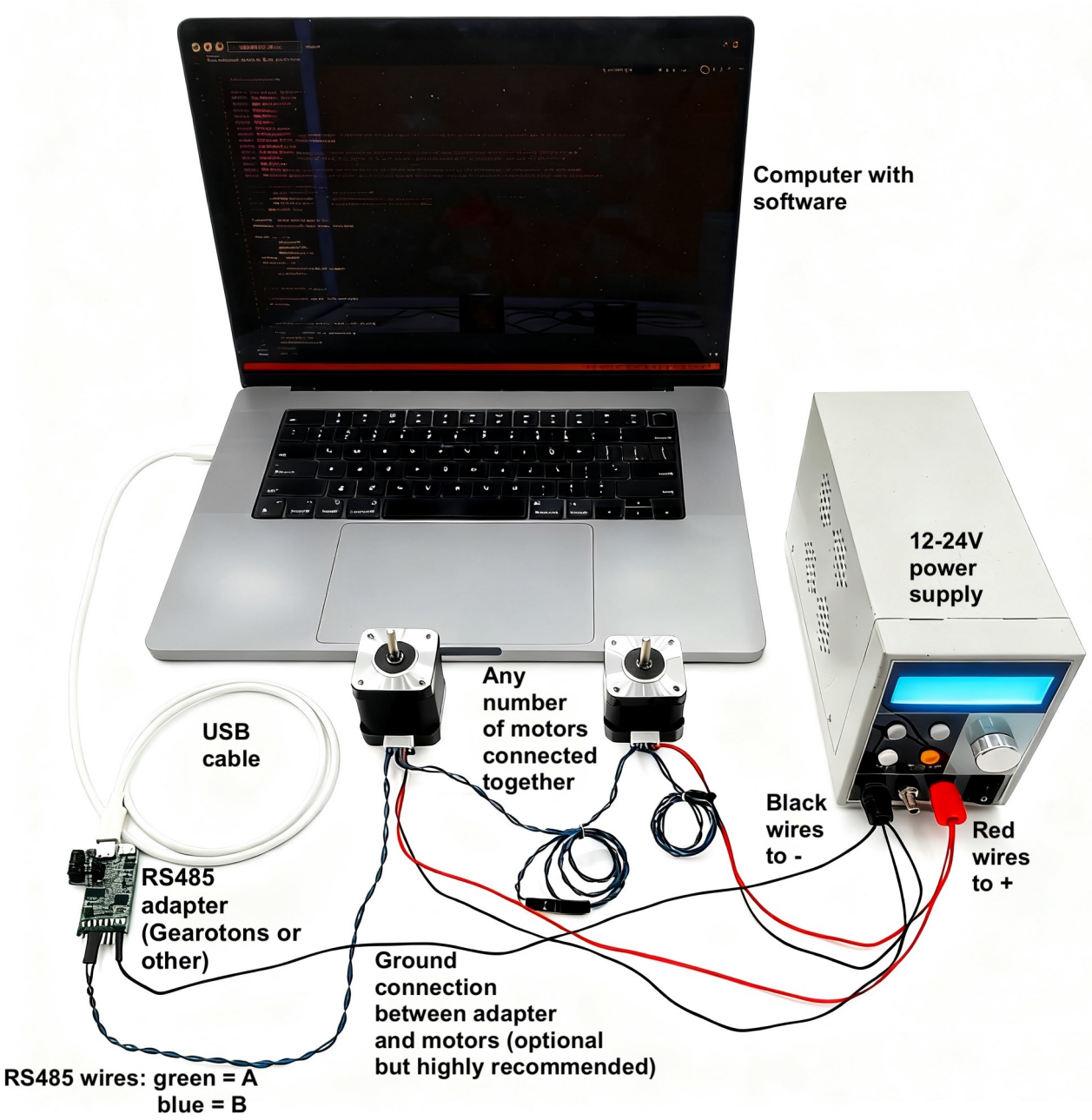
Wenn Sie Fehler in diesem Dokument finden oder Fragen haben, teilen Sie uns dies bitte unter tutorial.gearotons.com/feedback mit. Wir freuen uns sehr über Ihre Vorschläge, wie wir das Produkt oder die Dokumentation verbessern können. Vielen Dank!

Besuchen Sie unsere Feedback-Seite

Wichtigste Merkmale

- Hoher Integrationsgrad: Motor, Motortreiber, Bewegungssteuerungssystem und Encoder in einem Gerät
- Steuerung beliebig vieler Motoren über eine RS-485-Kommunikationsschnittstelle mit nur einem einfachen Controller
- Steuerung über High-Level-Befehle wie "Enable mosfets" (MOSFETs aktivieren) und "Trapezoid move" (Trapezbewegung) - keine zeitkritische DIR/STEP-Ansteuerung erforderlich
- Kompakte Bauform, nahezu gleich groß wie ein NEMA-17-Schrittmotor mit denselben Spezifikationen (keine überstehenden Teile)
- Standardisierte NEMA-17-Befestigungsmaße
- Weiter Spannungsbereich (12-24V) für flexible Stromversorgungsoptionen
- Hochpräzise Closed-Loop-Regelung mit integriertem Encoder und PID-Regelkreis mit 32 kHz Regelfrequenz
- Deutlich energieeffizienter im Closed-Loop-Betrieb als ein herkömmlicher Schrittmotor
- Integrierter Überstrom-, Überspannungs- und Übertemperaturschutz
- Maximale Drehzahl bis zu 560 RPM
- Drehmoment-Gewichts-Verhältnis entspricht dem eines vergleichbaren Schrittmotors
- Kompatibel mit einer Vielzahl von Schnittstellen und Hardware, z. B. Raspberry Pi, Arduino, ESP32, Mac und PC
- Wir bemühen uns um hervorragende Dokumentation und Tutorials, damit Sie schnell einsatzbereit sind
- Wir stellen KI-freundliche Dokumentation bereit, falls Ihre bevorzugte KI die ganze Arbeit für Sie erledigen soll
- Geeignet für Robotik, CNC, Automatisierung, wissenschaftliche Instrumente, Prüfvorrichtungen, 3D-Drucker und vieles mehr
- In verschiedenen Größen erhältlich, damit Sie das passende Drehmoment und den passenden Preis für Ihre Anwendung finden

Anschlussdiagramm



Einheitensystem

Die Servomotoren der M17-Serie verwenden bestimmte interne Einheiten, um die mit der Bewegung verbundenen Berechnungen effizient (mit Ganzzahlarithmetik) durchführen zu können. Es liegt in der Verantwortung der steuernden Software, mehrere Maßeinheiten für die verschiedenen Größen zu unterstützen. Unsere Python- und Arduino-Bibliotheken führen Einheitenumrechnungen automatisch durch, sodass Sie mit Ihren bevorzugten Einheiten arbeiten können. Nachfolgend sind die unterstützten Einheiten für jede Größe aufgeführt:

Größe	Verfügbare Einheiten
Zeit	seconds, milliseconds, minutes, microseconds, timesteps
Position	shaft rotations, degrees, radians, encoder counts
Geschwindigkeit	rotations per second, rpm, degrees per second, radians per second, counts per second, counts per timestep
Beschleunigung	rotations per second squared, rpm per second, degrees per second squared, radians per second squared, counts per second squared, counts per timestep squared
Strom	internal current units, milliamps, amps
Spannung	volts, millivolts
Temperatur	celsius, fahrenheit, kelvin

Einstiegsanleitung

Um Ihnen den Einstieg mit Ihrem Servomotor der M17-Serie zu erleichtern, stellen wir eine umfassende Online-Anleitung bereit, die alles von der Ersteinrichtung bis zu fortgeschrittenen Protokollimplementierungen abdeckt. Diese Anleitung umfasst:

- Schritt-für-Schritt-Anleitung zur Einrichtung
- Ausführliche Dokumentation des Kommunikationsprotokolls
- Programmierbeispiele und Codeausschnitte
- Beschreibung der Fehlercodes
- Tipps zur Fehlerbehebung und bewährte Vorgehensweisen



Klicken Sie hier für die Einstiegsanleitung: tutorial.gearotons.com

Status-LEDs und Taster

Der Servomotor verfügt über zwei Status-LEDs (grün und rot). Die grüne LED blinkt langsam als Herzschlagsignal, blinkt schnell, um anzuzeigen, dass der Bootloader anstelle der Anwendung läuft, und leuchtet außerdem kurz auf, während ein Paket empfangen wird, sodass sie bei Kommunikation auf dem Bus flackert. Die rote LED signalisiert fatale Fehlercodes durch eine bestimmte Anzahl von Blinkimpulsen.

Der Servomotor verfügt über zwei Taster mit den Bezeichnungen "Reset" und "Test". Der Reset-Taster setzt den internen Mikrocontroller zurück, und alle Zustände kehren zu den Standardwerten zurück. Der Test-Taster lässt den Motor drehen. Drücken Sie kurz, damit er in die eine Richtung dreht; drücken Sie länger als 0,3 Sekunden und lassen Sie los, damit er in die andere Richtung dreht. Halten Sie den Taster mindestens 2 Sekunden gedrückt und lassen Sie los, damit der Motor in den Closed-Loop-Modus wechselt. Halten Sie ihn länger als 15 Sekunden gedrückt und lassen Sie los, damit der Motor eine Selbstkalibrierung durchführt. Beachten Sie, dass sich der Motor während der Kalibrierung dreht und sich frei drehen können muss, damit die Kalibrierung erfolgreich ist. Entfernen Sie daher vor diesem Vorgang alle Lasten.

Überblick über das Kommunikationsprotokoll

Die Servomotoren der M17-Serie kommunizieren über RS-485, sodass mehrere Motoren in Reihe geschaltet (Daisy-Chain) werden können. Jeder Motor besitzt eine werkseitig zugewiesene, eindeutige 64-Bit-Adresse, über die er auf einem RS-485-Bus individuell angesprochen werden kann. Jedem Motor kann außerdem ein eindeutiger 1-Byte-Alias (0-251) zugewiesen werden, um Bytes im Kommunikationspaket zu sparen und dennoch eine vergleichbare individuelle Steuerung zu ermöglichen. Es gibt zudem einen Broadcast-Alias mit dem Wert 255. Das Protokoll unterstützt einen umfassenden Befehlssatz für Bewegungssteuerung, Konfiguration und Statusüberwachung. Firmware-Updates über die RS-485-Schnittstelle werden unterstützt. Die Baudrate der Kommunikation ist fest auf 230400 eingestellt.

Zusammenfassung der Befehlsreferenz

Die aktuelle, verbindliche Referenz aller verfügbaren Befehle finden Sie in diesem Dokument.

https://github.com/tomrodinger/servomotor/blob/main/python_programs/servomotor/motor_commands.json

Sie können auch diesen Befehl ausführen:

```
pip3 install servomotor   # run this just once to install the library and programs
servomotor_command -c
```

Dadurch werden die in der Datei motor_commands.json enthaltenen Informationen übersichtlich ausgegeben, zusammen mit Hinweisen zum Senden von Befehlen an den Motor über die Kommandozeile.

Grundsteuerung

Befehl	Beschreibung
Disable MOSFETs	Deaktiviert sofort die Motortreiber-Ausgänge (MOSFETs); der Befehl wird sofort ausgeführt (nicht in die Warteschlange gestellt) und ist idempotent.
Enable MOSFETs	Aktiviert die Motortreiber-Ausgänge (MOSFETs).
Reset time	Setzen Sie die absolute Mikrosekundenuhr des Geräts auf null zurück; als wesentliche Nebenwirkung wird die gesamte Bewegungswarteschlange geleert und der Motor sofort angehalten.
Emergency stop	Stoppen Sie den Motor sofort: Die MOSFETs werden deaktiviert, die Bewegungswarteschlange geleert, die aktuelle Geschwindigkeit auf null gesetzt und der interne Positionssollwert auf die aktuelle Position gesetzt.
Zero position	Macht die aktuelle Position zur Nullposition (dem Ursprung).
System reset	Setzt das Gerät zurück.

Bewegungssteuerung

Befehl	Beschreibung
Trapezoid move	Fährt die angegebene vorzeichenbehaftete Verschiebung in genau der angegebenen Dauer, relativ zur Position am Ende der zuvor in die Warteschlange gestellten Bewegung (für ein absolutes Ziel verwenden Sie 'Go to position').
Go to position	Stellt eine gleichmäßige absolute Bewegung in die Warteschlange.
Homing	Führt eine Referenzfahrt des Motors durch, indem er bis zum Anschlag fährt.
Go to closed loop	Wechselt in den Closed-Loop-Positionsregelmodus.
Move with acceleration	Stellt ein Segment mit konstanter Beschleunigung in die Warteschlange: Die Beschleunigung wird einmal pro Zeitschritt für die angegebene Anzahl von Zeitschritten angewendet.
Move with velocity	Dreht den Motor für eine festgelegte Dauer mit konstanter Geschwindigkeit.
Multimove	Stellt bis zu 32 Bewegungen mit einem einzigen Befehl in die Warteschlange; jede Bewegung ist ein Beschleunigungssegment oder eine sofortige Geschwindigkeitsänderung (pro Bewegung durch die moveTypes-Bitmaske ausgewählt), die für ihre angegebene Anzahl von Zeitschritten ausgeführt wird.

Konfiguration

Befehl	Beschreibung
Set maximum velocity	Stellt die maximal zulässige Geschwindigkeit ein.
Set maximum acceleration	Stellt die maximale Beschleunigung ein, die zur Planung aller danach in die Warteschlange gestellten Trapezbewegungen verwendet wird ('Go to position', 'Trapezoid move', Kalibrierungsbewegungen); bereits in der Warteschlange befindliche Einträge sind davon nicht betroffen.
Start calibration	Führt eine vollständige Motorkalibrierung durch.
Set maximum motor current	Stellt den maximalen Motorstrom und den maximalen Rekuperationsstrom ein.
Set safety limits	Stellt die Positions-Sicherheitsgrenzen ein.
Test mode	Stellt einen Testmodus ein oder löst ihn aus.
Set PID constants	Stellt die P-, I- und D-Konstanten des Closed-Loop-Reglers ein, der die Bewegungstrajektorie einzuhalten versucht.
Set max allowable position deviation	Legen Sie den maximalen Abstand fest, um den die tatsächliche Motorposition (gemessen von den Hall-Sensoren) von der Sollposition abweichen darf; eine Überschreitung löst den fatalen Fehler 45 (ERROR_POSITION_DEVIATION_TOO_LARGE) aus.
CRC32 control	Aktiviert oder deaktiviert die CRC32-Schicht des Protokolls für dieses Gerät in beiden Richtungen.
Get PID constants	Lesen Sie einen konsistenten Schnappschuss der konfigurierten rohen P-, I- und D-Konstanten.

Status und Überwachung

Befehl	Beschreibung
Get current time	Geben Sie die absolute Gerätezeit als vorzeichenlose 64-Bit-Anzahl tatsächlicher Mikrosekunden seit dem Einschalten oder dem letzten 'Reset time' zurück; dies ist keine Kalenderzeit.
Get n queued items	Geben Sie die Anzahl der Elemente in der Bewegungswarteschlange als einzelnes Byte zwischen 0 und 32 zurück (die Warteschlange fasst höchstens 32 Elemente).
Get hall sensor position	Liest die von den Hall-Sensoren gemessene Position aus, die der tatsächlichen Wellenposition entspricht.
Get status	Liest die Status-Flags und den fatalen Fehlercode des Motors aus.
Control hall sensor statistics	Schaltet die Erfassung der Hall-Sensor-Statistiken ein oder aus.
Get hall sensor statistics	Liest die von den drei analogen Hall-Sensoren erfassten Statistiken aus.
Get position	Liest die aktuelle Soll- (gewünschte) Position aus, d. h. die interne Zielposition des Bewegungsprofils, die mit jedem Regeltakt fortschreitet.
Get comprehensive position	Liest die Sollposition, die (gemessene) Hall-Sensor-Position und den Zählerstand des externen Encoders in einem Zugriff aus.
Get supply voltage	Liest die gemessene Versorgungsspannung aus.
Get max PID error	Liest den minimalen und maximalen Positionsfehler aus, der seit dem letzten Auslesen vom Closed-Loop-PID-Regler beobachtet wurde, und setzt anschließend die Akkumulatoren zurück.
Get temperature	Liest die Temperatur aus, die von einem dedizierten analogen Sensor auf der Motortreiber-Platine gemessen wird (nicht des Mikrocontroller-Dies und nicht der Motorwicklungen).
Get debug values	Lesen Sie einen Schnappschuss zahlreicher interner Diagnosewerte.
Get communication statistics	Liest sechs Kommunikationsfehlerzähler aus und setzt sie optional zurück.

Geräteverwaltung

Befehl	Beschreibung
Time sync	Senden Sie den Zeitwert des Masters an den Motor, damit dieser seine Taktrate nachregeln kann.
Get product specs	Liest zwei Produktkonstanten aus, die für Bewegungsberechnungen benötigt werden.
Detect devices	Erkennt alle an der RS485-Schnittstelle angeschlossenen Geräte.
Set device alias	Stellt den Ein-Byte-Alias des Geräts ein, speichert ihn im nichtflüchtigen Flash-Speicher und setzt das Gerät anschließend automatisch zurück.
Get product info	Liest die Produktinformationen aus: Produktcode (Modellnummer), Firmware-Kompatibilitätscode, Hardware-Version, Seriennummer und eindeutige ID, zurückgegeben als 28-Byte-Antwortnutzlast.
Firmware upgrade	Schreibt eine 2048-Byte-Seite der Anwendungsfirmware in den Flash-Speicher.
Get product description	Liest eine kurze Zeichenfolge mit der Produktbeschreibung aus.
Get firmware version	Liest die Firmware-Version aus, oder die Bootloader-Version, wenn auf dem Gerät derzeit der Bootloader läuft.
Ping	Sendet eine 10-Byte-Nutzlast, und das Gerät antwortet sofort mit denselben 10 Bytes (wird sofort ausgeführt, nicht in die Warteschlange gestellt).
Vibrate	Lassen Sie den Motor vibrieren (nur bei M1 implementiert; bei M2, M17 und M23 ohne Wirkung), indem die Open-Loop-PWM-Spannung schnell zwischen +50 und -50 wechselt (feste, fest codierte Amplitude), oder beenden Sie die Vibration.
Identify	Identifiziert einen Motor, indem seine grüne LED schnell blinkt: 30 Blinkimpulse zu je etwa 90 ms, insgesamt etwa 2,7 Sekunden, danach wird das normale Herzschlag-Blinken im 1-Sekunden-Takt fortgesetzt.

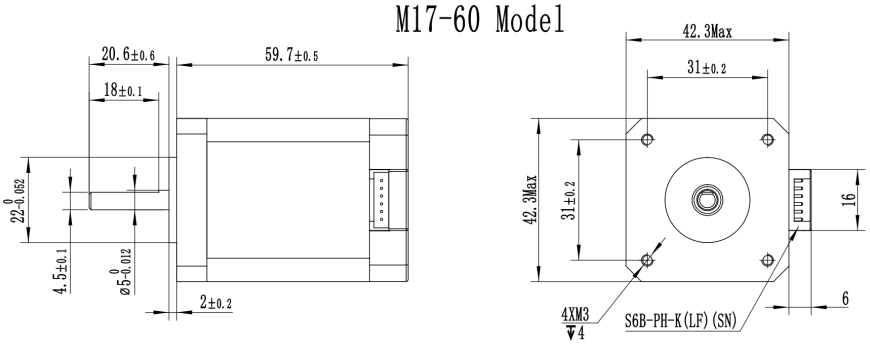
Sonstiges

Befehl	Beschreibung
Capture hall sensor data	Erfasst Hall-Sensor-Daten und streamt sie zurück.
Read multipurpose buffer	Liest den Mehrzweck-Datenpuffer aus und leert ihn; er fasst jeweils höchstens einen Datensatz.

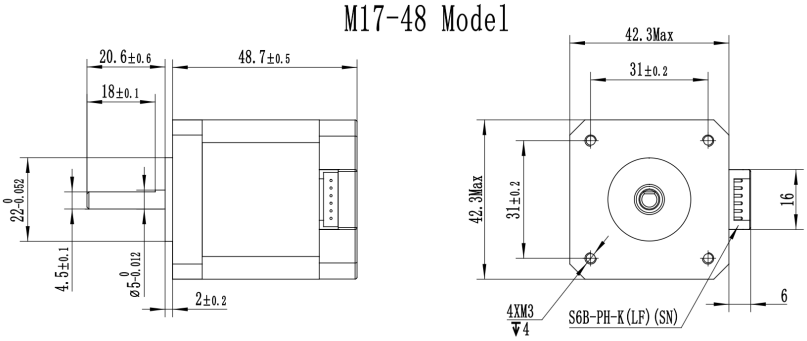
Mechanische Spezifikationen

Parameter	M17-60	M17-48	M17-40	M17-34
Abmessungen (LxB)	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm
Höhe	59.7 mm	48.7 mm	40.1 mm	33.5 mm
Wellenlänge	20.6 mm	20.6 mm	20.6 mm	20.6 mm
Gewicht	470g	360g	285g	210g
Schutzart	IP20	IP20	IP20	IP20

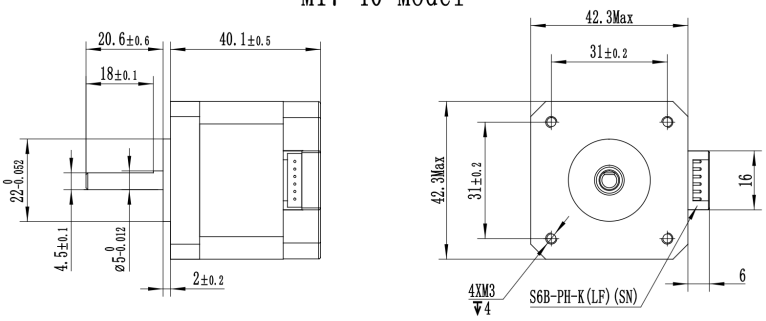
M17-60 Model



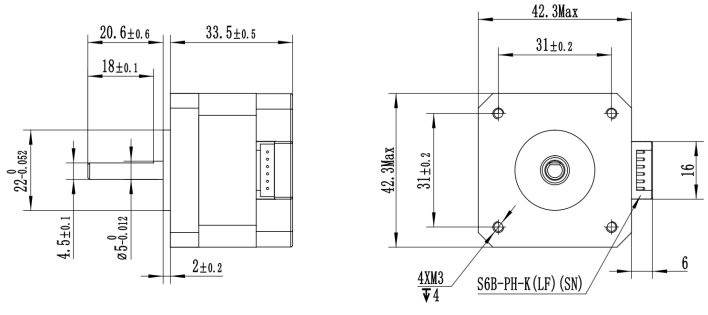
M17-48 Model



M17-40 Model



M17-34 Model



Technische Spezifikationen

Parameter	M17-60	M17-48	M17-40	M17-34
Betriebsspannung	12-24V	12-24V	12-24V	12-24V
Nenndrehmoment	0.65 N.m	0.55 N.m	0.42 N.m	0.28 N.m
Maximale Drehzahl	560 RPM	560 RPM	560 RPM	560 RPM
Maximaler Strom	1.1A	1.1A	1.1A	1.0A
Nennleistung	26.4W	26.4W	26.4W	24.0W

Betriebsbedingungen

Parameter	Spezifikation
Betriebstemperatur	0C bis +80C
Lagertemperatur	-20C bis +60C
Luftfeuchtigkeitsbereich	20% bis 80% RH (nicht kondensierend)
Einsatzumgebung	Nur für den Innenbereich

Python-Bibliothek

Unsere Python-Bibliothek bietet eine umfassende Schnittstelle zur Steuerung der Servomotoren der M17-Serie. Die Bibliothek übernimmt alle Low-Level-Kommunikationsprotokolle und Einheitenumrechnungen und erleichtert so die Integration der Servomotorsteuerung in Ihre Python-Projekte.

Installation

Installieren Sie die servomotor-Bibliothek mit pip:

```
pip3 install servomotor
```

Beispiel: Grundlegende Motorsteuerung

Das folgende Beispiel zeigt, wie Sie eine Verbindung zu einem Servomotor herstellen, die MOSFETs aktivieren und eine Trapezbewegung ausführen:

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X'                               # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Serial device path; change if needed (e.g., "COM3" on
                                          #  Windows)
DISPLACEMENT_ROTATIONS = 1.0              # 1 rotation
DURATION_SECONDS = 1.0                    # 1 second
DELAY_MARGIN = 0.10                       # +10% wait margin because the motor's clock is not
                                          #  perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
                                          #  should prompt you for the serial port or it will use
                                          #  the last used port from a file
servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Wichtigste Merkmale

- Unterstützung aller Motorbefehle
- Automatische Einheitenumrechnung für Zeit, Position, Geschwindigkeit, Beschleunigung, Strom, Spannung und Temperatur
- Fehlerbehandlung und Statusüberwachung
- Unterstützung mehrerer Motoren am selben Bus
- Einsatzbereit mit nur wenigen Zeilen Code

Dokumentation

Die vollständige API-Dokumentation und weitere Beispiele finden Sie unter:
https://github.com/tomrodinger/servomotor/tree/main/python_programs

Arduino-Bibliothek

Die Arduino-Bibliothek für die Servomotoren der M17-Serie bietet eine einfach zu bedienende Schnittstelle zur Steuerung der Motoren von Arduino-Boards aus. Die Bibliothek unterstützt serielle Hardware-Kommunikation und enthält alle wesentlichen Funktionen zur Motorsteuerung.

Installation

Installieren Sie die Bibliothek über den Arduino-Bibliotheksverwalter:

- 1. Öffnen Sie die Arduino IDE
- 2. Gehen Sie zu Sketch → Bibliothek einbinden → Bibliotheken verwalten
- 3. Suchen Sie nach "Servomotor"
- 4. Suchen Sie die Bibliothek mit dem Namen "Servomotor by Gearotons" und klicken Sie auf Installieren



Manuelle Installation

Alternativ können Sie als erfahrener Anwender den Code der Arduino-Bibliothek hier herunterladen: https://github.com/tomrodinger/Servomotor_Arduino_Library

Beispiel: Grundlegende Motorsteuerung

Dieses Beispiel zeigt, wie Sie einen Motor initialisieren, die MOSFETs aktivieren und eine Trapezbewegung ausführen:

```
// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
//  system reset -> enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds)
//  -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.
// - Every command is checked with getError(). No command method returns a success
//   flag, and a motor that has latched a fatal error silently ignores everything
//   afterwards, so an unchecked sketch just stops moving with no symptom at all.
//   See "Checking for errors" in the API documentation.

#include <Servomotor.h>

#define ALIAS 'X'                   // Device alias
#define BAUD 230400                 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f       // 1 second
#define TOLERANCE_PERCENT 10        // +10% wait margin because the motor's clock is not
                                    //  perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))
#define POST_RESET_WAIT_MS 1500     // Keep the bus SILENT this long after a reset: the motor
                                    //  boots through a bootloader window, and any packet that
                                    //  arrives during it pins the motor in the bootloader.

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4             // TX pin to RS485 transceiver
#define RS485_RXD 5             // RX pin from RS485 transceiver
#endif

// Returns true (and explains itself) if the previous command did not succeed.
// getError() is 0 on success, positive for a fatal error reported by the motor
// (look the number up in the Error Codes section of the documentation), and
// negative for a communication failure such as -1 = no reply within 1 second.
bool failed(Servomotor &motor, const char *what) {
  int e = motor.getError();
  if (e == 0) return false;
  Serial.print("[FAIL] ");
  Serial.print(what);
  Serial.print(" -> getError() = ");
  Serial.print(e);
  Serial.println(e > 0 ? "  (motor fatal error - see the Error Codes section)"
                       : "  (communication failure - check wiring, alias and power)");
  return true;
}

void setup() {
  Serial.begin(115200); // Console serial for debugging
                        // On ESP32-S3, set Tools > USB CDC On Boot > Enabled or this
                        // output never reaches the USB serial monitor.

  // Create the motor; serial port opens on first instantiation.
#if defined(ESP32)
  Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
#else
  Servomotor motor(ALIAS, Serial1);
#endif

  // Use units: rotations for position, seconds for time.
  // These are host-side only - they send nothing to the motor.
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  // Start from a known-clean state: this also clears any fatal error left over
  // from a previous run, which is the usual reason a motor "stops working".
  motor.systemReset();
  if (failed(motor, "systemReset")) return;
  delay(POST_RESET_WAIT_MS);

  motor.enableMosfets();
```

```
  if (failed(motor, "enableMosfets")) return;

  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  if (failed(motor, "trapezoidMove")) return;

  delay(WAIT_MS);

  // A successful move command only means the move was accepted and queued. Faults
  // such as a stall or a position-deviation trip happen later, while it executes,
  // so check the motor's own status once the motion should be finished.
  getStatusResponse status = motor.getStatus();
  if (failed(motor, "getStatus")) return;
  if (status.fatalErrorCode != 0) {
    Serial.print("[FAIL] motor faulted during the move, error code ");
    Serial.println(status.fatalErrorCode);
    return;
  }

  motor.disableMosfets();
  if (failed(motor, "disableMosfets")) return;

  Serial.println("Move completed successfully.");
}

void loop() {
}
```

Unsere Mission

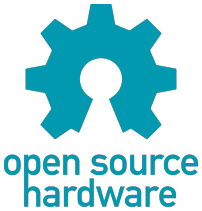
Wir bei Gearotons sind überzeugt, dass die Zukunft denjenigen gehört, die künstliche Intelligenz und Automatisierungssysteme verstehen und mit ihnen zusammenarbeiten können. Unsere Mission ist es, die nächste Generation von Innovatoren zu stärken, indem wir fortschrittliche Bewegungssteuerungstechnologie für Maker, Lehrende und Ingenieure weltweit zugänglich, verständlich und erschwinglich machen. Wir setzen uns dafür ein, die Barrieren abzubauen, die Studierende und Hobbyanwender traditionell von professionellen Automatisierungswerkzeugen getrennt haben. Mit integrierten Lösungen, die Komplexität beseitigen, ohne auf Leistungsfähigkeit zu verzichten, ermöglichen wir praxisnahe Lernerfahrungen, die junge Menschen auf eine Welt vorbereiten, in der menschliche Kreativität und maschinelle Präzision harmonisch zusammenwirken. Unsere Grundüberzeugung ist, dass jeder Studierende die Möglichkeit haben sollte, mit den Bausteinen der modernen Automatisierung zu experimentieren - von Robotik und CNC-Systemen bis hin zu wissenschaftlichen Instrumenten. Durch unsere Bildungspartnerschaften und unseren Open-Source-Ansatz fördern wir eine Generation, die Technologie nicht nur nutzt, sondern sie wirklich versteht und gestaltet. Gemeinsam schaffen wir das Fundament für Innovatoren, die die großartige Zukunft prägen werden.

Open Source

Wir glauben daran, die Welt durch Technologie besser zu machen. Software, Firmware und PCB-Designdateien sind hier verfügbar:



https://github.com/tomrodinger/servomotor





Datenblatt-Version: 1.11 Veröffentlichungsdatum: Oct. 29, 2026

(c) 2026 Alle Spezifikationen können ohne Vorankündigung geändert werden.